

Name: Chirag Das

Reg No: 2648517

Exercise 2: Include and Demonstrate HTML5 APIs

Project: **We Rise**, a productivity landing page (Lab 1), extended with a working “Start Today” section

1. Problem Statement

Enhance the existing landing page by integrating HTML5 APIs to create a more interactive user experience. After the story and glass-shard sections of We Rise, I added **Start Today**: a small working version of the app with three tools. Each tool uses real HTML5 APIs that fit the product idea (start the day early, write down your tasks, and focus).

2. HTML5 APIs Used (8)

API	Where	What it does on the page
Geolocation API	Where you Rise	“Use my location” gets your latitude and longitude, then shows your place, next sunrise and weather.
Fetch API	Where you Rise	Calls free web services: Open-Meteo (sunrise, weather, city search) and BigDataCloud (place name).
Canvas API (WebGL, via Three.js)	Where you Rise, Focus soundscape	Draws a real 3D Earth lit by the sun as it is right now. A 2D canvas draws the sound bars.
Web Speech API	Speak a quest	Speech to text: say a task and it is added to your list.
Media Capture and MediaRecorder API	Speak a quest	Records the microphone when a browser has no speech service (e.g. Brave); the audio is turned into text on the device.
Audio API (HTML5 audio)	Speak a quest	Plays a voice reply (“Quest accepted.”) when a task is added or finished.
Local Storage API	Speak a quest, Where you Rise	Saves your tasks, chosen place, sound mode and volume, so they are still there after a refresh.
Web Audio API	Focus soundscape	Generates rain, brown noise and alpha waves live in the browser, with no audio files.

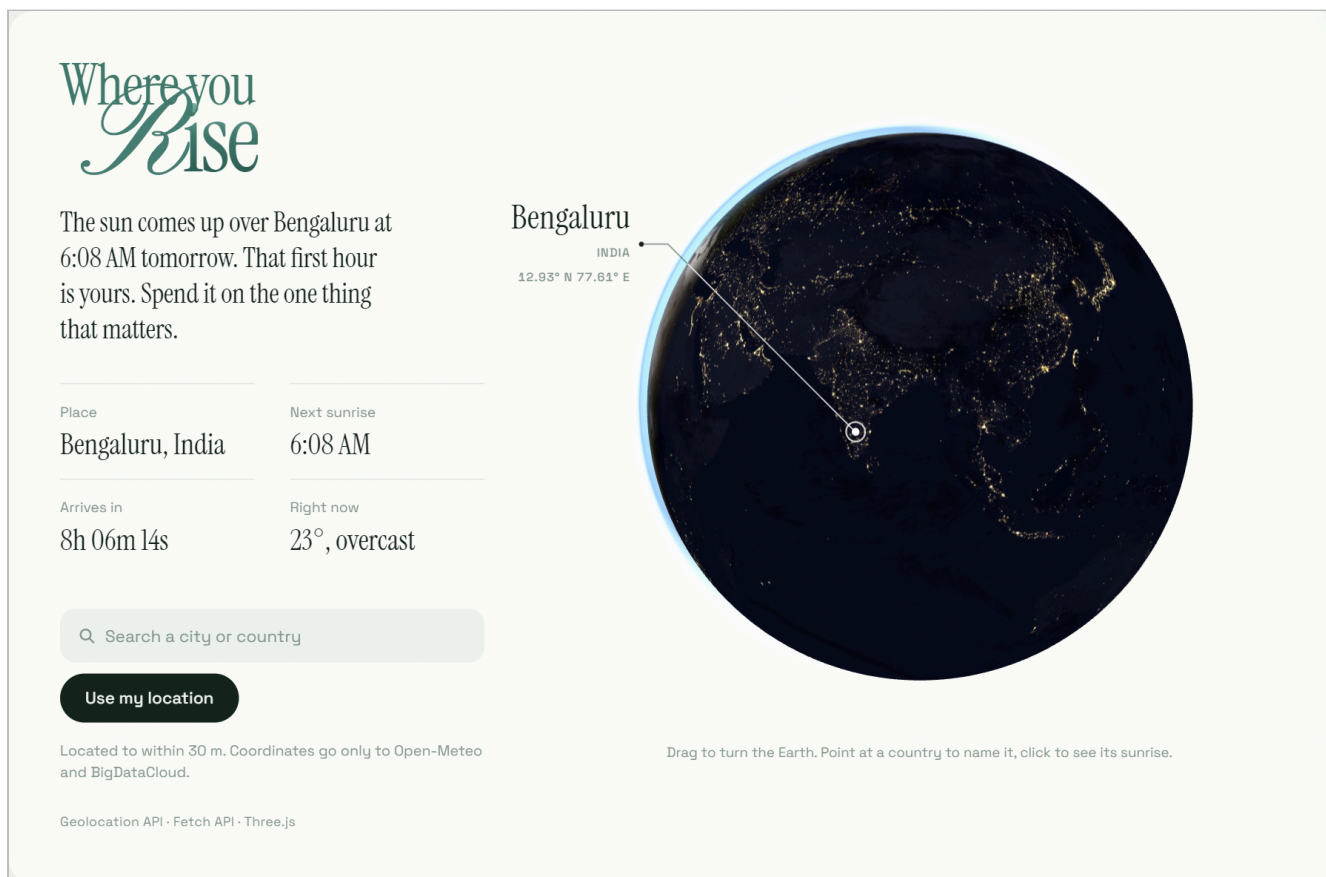
Software used: VS Code, Google Chrome / Microsoft Edge, Git, GitHub (Pages for deployment).

3. GitHub Repository

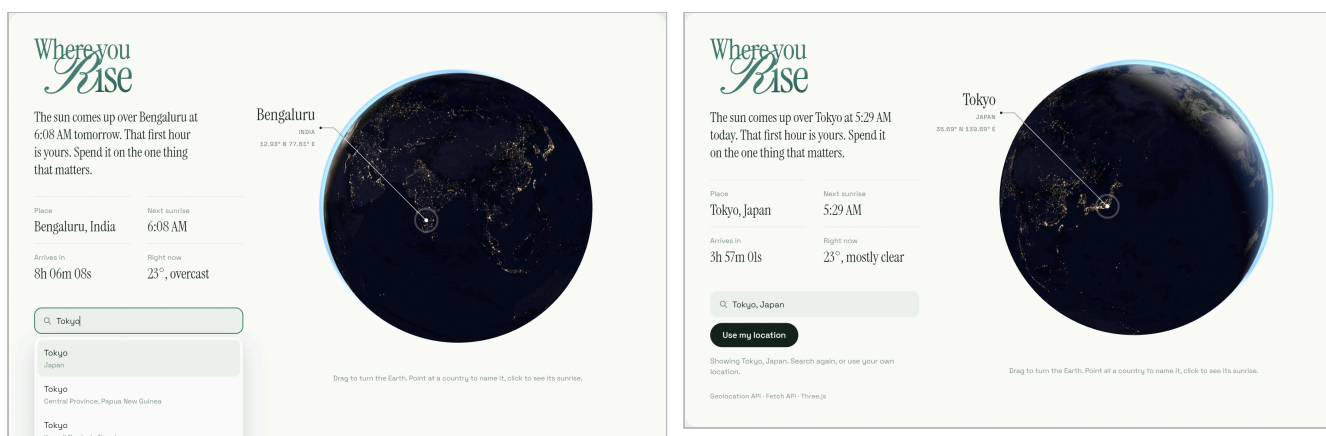
- Repository: <https://github.com/Chiragdascodes/Lab1FSD>
- Live website: <https://chiragdascodes.github.io/Lab1FSD/>

4. Screenshots (API Integrations)

4.1 Geolocation API + Fetch API + Canvas (WebGL)

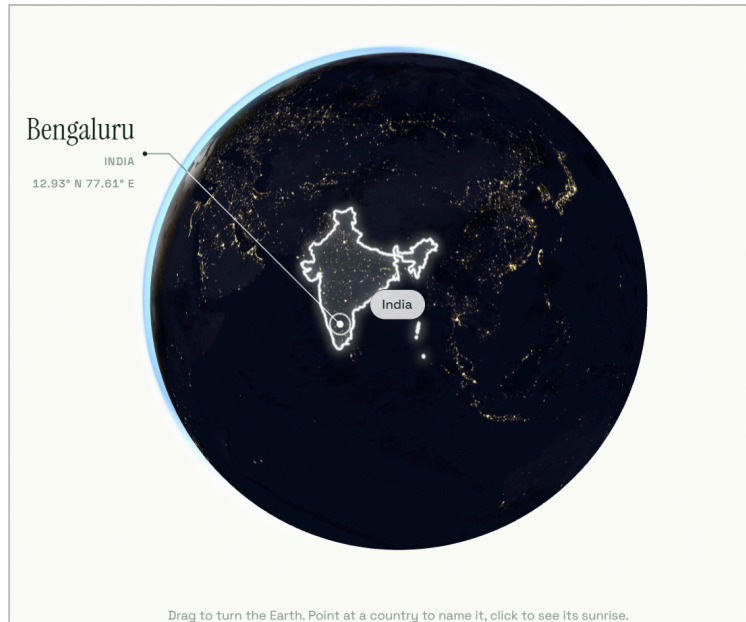


After “Use my location”: place name, next sunrise, live countdown and weather. The status line shows the accuracy (30 m). The 3D Earth turns to the location.



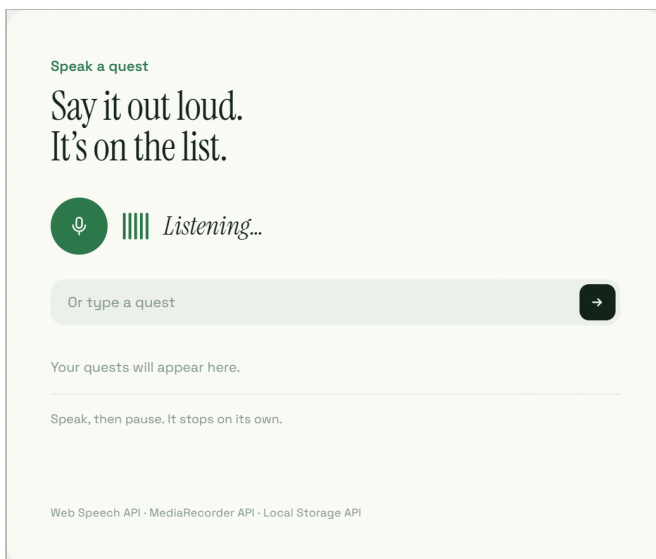
Typing a city calls the Open-Meteo geocoding service through Fetch.

Choosing Tokyo loads its sunrise and weather; the Earth shows it on the night side.

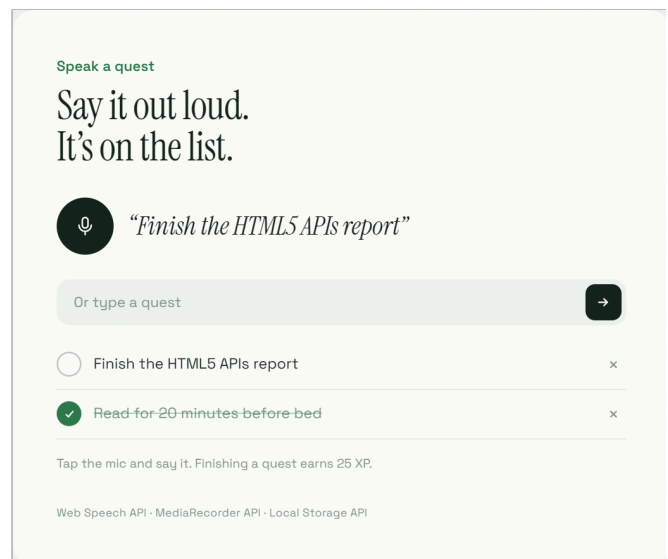


Pointing at a country outlines it in white and names it. The place label sits outside the Earth.

4.2 Web Speech API + MediaRecorder + Audio + Local Storage



Microphone on: the bars follow the real input level.

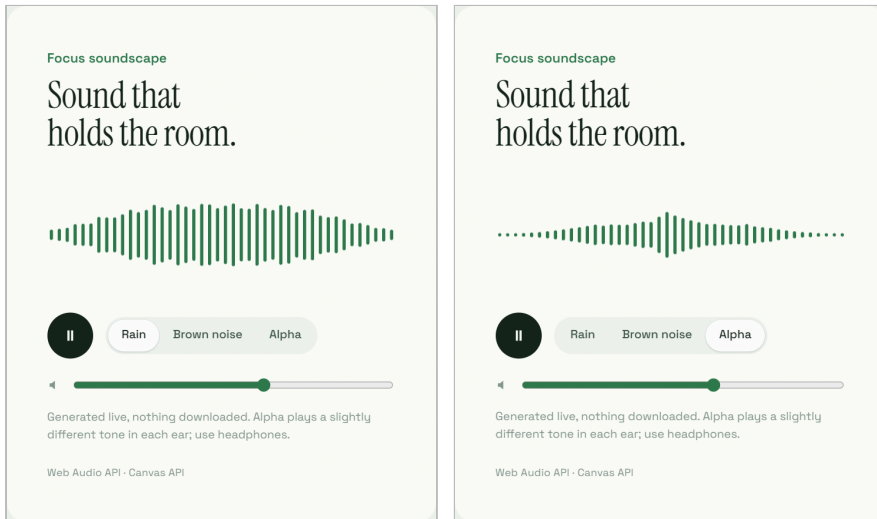


The spoken task was written down; a typed task was added and one marked done.

Data saved by the Local Storage API (key `wr-system-v2`):

```
{
  "quests": [
    {"id": "vmucw6vlo", "text": "Finish the HTML5 APIs report", "done": false},
    {"id": "vmucw6vla", "text": "Read for 20 minutes before bed", "done": true}
  ],
  "place": {"lat": 35.6895, "lon": 139.69171, "name": "Tokyo, Japan"},
  "mode": "rain", "volume": 0.6
}
```

4.3 Web Audio API + Canvas API



Rain, generated live. The Canvas draws the sound level.

Alpha: 200 Hz in the left ear, 210 Hz in the right.

5. Source Code

The full code is in the repository. Below are the new section of **index.html** and the key JavaScript for each API (from **js/system.js**).

5.1 index.html: the Start Today section

```
<section id="system" class="system" aria-labelledby="system-title">
  <div class="sys-wrap">

    <header class="sys-head">
      <h2 class="sys-title" id="system-title"><span class="t-sm">Start</span><span class="t-lg"><span
class="cap">T</span>oday</span></h2>
      <p class="sys-sub">The app in miniature, running right here in your browser.</p>
    </header>

    <div class="bento">

      <!-- where you rise -->
      <article class="tile tile-rise" aria-labelledby="rise-title">
        <div class="rise-copy">
          <h3 class="rise-title" id="rise-title"><span class="t-sm">Where you</span><span class="t-lg"><span
class="cap">R</span>ise</span></h3>
          <p class="rise-line" id="rise-line">Share your location and we'll find your sunrise: the first
hour of your day, and the one most worth guarding.</p>

          <dl class="specs">
            <div><dt>Place</dt><dd id="rise-place">&mdash;</dd></div>
            <div><dt>Next sunrise</dt><dd id="rise-sun">&mdash;</dd></div>
            <div><dt>Arrives in</dt><dd id="rise-in">&mdash;</dd></div>
            <div><dt>Right now</dt><dd id="rise-now">&mdash;</dd></div>
          </dl>

          <div class="place-controls">
            <div class="place-search" role="search">
              <div class="field place-field">
                <svg class="field-icon" viewBox="0 0 16 16" aria-hidden="true"><circle cx="7" cy="7" r="4.5" />
<path d="M10.4 10.4L14 14" /></svg>
                <label class="sr-only" for="place-input">Search a city or country</label>
                <input id="place-input" type="text" role="combobox" aria-autocomplete="list" aria-
expanded="false"
                    aria-controls="place-list" autocomplete="off" spellcheck="false" placeholder="Search a
city or country" />
              </div>
            </div>
          </div>
        </div>
      </article>
    </div>
  </div>
```

```

        <ul class="place-list" id="place-list" role="listbox" aria-label="Places" hidden></ul>
      </div>
      <button type="button" class="pill" id="rise-locate">Use my location</button>
    </div>
    <p class="tile-note" id="rise-status">Search any city or country, or use your own location. Coordinates
go only to Open-Meteo and BigDataCloud.</p>
  </div>

  <figure class="rise-globe" id="rise-globe">
    <div class="globe-stage">
      <canvas id="globe" role="img" aria-label="The Earth, lit by the sun where it is now, with your place
marked"></canvas>
      <svg class="globe-lead" aria-hidden="true"><path class="lead-in" pathLength="1"/><path class="lead-
out" pathLength="1"/><circle class="lead-end" r="2.25"/></svg>
      <p class="globe-tag" id="globe-tag" aria-hidden="true"><span class="gt-name"></span><span class="gt-
meta"></span><span class="gt-coord"></span></p>
      <span class="globe-hover" id="globe-hover" aria-hidden="true"></span>
    </div>
    <figcaption class="globe-cap">Drag to turn the Earth. Point at a country to name it, click to see its
sunrise.</figcaption>
  </figure>

  <p class="tile-apis">Geolocation API &middot; Fetch API &middot; Three.js</p>
</article>

<!-- speak a quest -->
<article class="tile tile-voice" aria-labelledby="vq-title">
  <p class="tile-label">Speak a quest</p>
  <h3 class="tile-head" id="vq-title">Say it out loud. It&rsquo;s on the list.</h3>

  <div class="vq-stage">
    <button type="button" class="vq-mic" id="vq-mic" aria-pressed="false" aria-label="Speak a quest">
      <svg viewBox="0 0 24 24" aria-hidden="true"><rect x="9" y="3" width="6" height="11" rx="3" /><path
d="M5.5 11a6.5 6.5 0 0 0 13 0M12 17.5V21" /></svg>
    </button>
    <div class="vq-meter" aria-hidden="true"><i></i><i></i><i></i><i></i><i></i><i></i><i></i><i></i></div>
    <p class="vq-live" id="vq-live" aria-live="polite">&ldquo;Read for twenty minutes&rdquo;</p>
  </div>

  <form class="field" id="vq-type" autocomplete="off">
    <label class="sr-only" for="vq-input">Or type a quest</label>
    <input id="vq-input" type="text" maxLength="60" placeholder="Or type a quest" />
    <button type="submit" class="field-go" aria-label="Add quest">
      <svg viewBox="0 0 16 16" aria-hidden="true"><path d="M3 8h9M8.5 4.5L12 8l-3.5 3.5" /></svg>
    </button>
  </form>

  <ol class="vq-list" id="vq-list" aria-label="Your quests"></ol>
  <p class="tile-note" id="vq-status">Tap the mic and say it. Finishing a quest earns 25 XP.</p>
  <p class="tile-apis">Web Speech API &middot; MediaRecorder API &middot; Local Storage API</p>
</article>

<!-- focus soundscape -->
<article class="tile tile-sound" aria-labelledby="fs-title">
  <p class="tile-label">Focus soundscape</p>
  <h3 class="tile-head" id="fs-title">Sound that holds the room.</h3>

  <div class="fs-stage">
    <canvas id="fs-viz" aria-hidden="true"></canvas>
  </div>

  <div class="fs-row">
    <button type="button" class="fs-play" id="fs-play" aria-pressed="false" aria-label="Play the
soundscape">
      <svg class="i-play" viewBox="0 0 24 24" aria-hidden="true"><path d="M8 5.5v13l11-6.5z" /></svg>
      <svg class="i-pause" viewBox="0 0 24 24" aria-hidden="true"><path d="M8 5h3v14H8zM13 5h3v14h-3z" />
</svg>
    </button>
    <div class="segmented" role="radiogroup" aria-label="Soundscape">
      <button type="button" role="radio" data-mode="rain" aria-checked="true">Rain</button>
      <button type="button" role="radio" data-mode="brown" aria-checked="false">Brown noise</button>
      <button type="button" role="radio" data-mode="alpha" aria-checked="false">Alpha</button>
    </div>

```

```

    </div>
    <label class="fs-vol"><span class="sr-only">Volume</span>
      <svg viewBox="0 0 16 16" aria-hidden="true"><path d="M2.5 6h2.5L8.5 3v10L5 10H2.5z" /></svg>
      <input type="range" id="fs-vol" min="0" max="1" step="0.01" value="0.6" />
    </label>
    <p class="tile-note" id="fs-status">Generated live, nothing downloaded. Alpha plays a slightly different
tone in each ear; use headphones.</p>
    <p class="tile-apis">Web Audio API &middot; Canvas API</p>
  </article>

</div>
</div>
</section>

```

5.2 Geolocation API

```

navigator.geolocation.getCurrentPosition((pos) => {
  riseStatus.textContent = 'Located to within ' + Math.round(pos.coords.accuracy) + ' m.';
  loadPlace({ lat: pos.coords.latitude, lon: pos.coords.longitude }, true);
}, (err) => {
  riseStatus.textContent = err.code === 1 ? 'Location is blocked for this site.' : 'Try again.';
}, { enableHighAccuracy: false, timeout: 12000, maximumAge: 60000 });

```

5.3 Fetch API

```

const weather = fetch('https://api.open-meteo.com/v1/forecast?latitude=' + lat + '&longitude=' + lon +
  '&current=temperature_2m,weather_code&daily=sunrise,sunset&timezone=auto&forecast_days=2')
  .then((r) => r.json());

// city or country search, cancelled if the user keeps typing
fetch('https://geocoding-api.open-meteo.com/v1/search?name=' + encodeURIComponent(q) + '&count=6',
  { signal: placeAbort.signal }).then((r) => r.json()).then((d) => showPlaces(d.results || []));

```

5.4 Local Storage API

```

const KEY = 'wr-system-v2';
const state = Object.assign({ quests: [], place: null, mode: 'rain', volume: 0.6 },
  JSON.parse(localStorage.getItem(KEY)) || {});
function save() { localStorage.setItem(KEY, JSON.stringify(state)); }

```

5.5 Web Speech API and MediaRecorder

```

const SR = window.SpeechRecognition || window.webkitSpeechRecognition;
const rec = new SR();
rec.lang = 'en-GB'; rec.interimResults = true;
rec.onresult = (e) => { heard = [...e.results].map((r) => r[0].transcript).join(''); };
rec.onend = () => { if (heard.trim()) addQuest(heard); };
rec.start();

// fallback: record the microphone, then transcribe on the device
stream = await navigator.mediaDevices.getUserMedia({ audio: true });
recorder = new MediaRecorder(stream);
recorder.ondataavailable = (e) => chunks.push(e.data);
recorder.onstop = () => transcribe(new Blob(chunks, { type: recorder.mimeType }));
recorder.start();

```

5.6 Audio API (voice reply)

```

const a = new Audio('assets/voice/accepted-1.mp3'); // "Quest accepted."
a.play();

```

5.7 Web Audio API and Canvas API

```

// binaural alpha: a 10 Hz beat between the ears
[[200, -1], [210, 1]].forEach(([f, pan]) => {
  const o = ctx.createOscillator(); o.frequency.value = f;

```

```

const p = ctx.createStereoPanner(); p.pan.value = pan;
o.connect(p); p.connect(gain); o.start();
});
analyser.getByteFrequencyData(bins); // read the level
vctx.fillRect(x, mid - h / 2, barWidth, h); // draw each bar on the canvas

```

6. AI Usage Declaration

AI tool	Purpose of use
Claude (Anthropic)	Help with writing and debugging code, about 40% of the JavaScript and CSS.

My own work: the idea and the product concept, the UI/UX design in Figma, the colour palette, the layout of the three tools, the choice of APIs and how each one fits the domain, the copy on the page, testing, and deployment on GitHub Pages.

7. Self Learning

Everything below goes beyond the lab guide. I learned it myself for this project:

- **Three.js and WebGL shaders:** I wrote custom GLSL shaders for a realistic Earth, using day and night imagery, city lights, clouds, ocean glint and a thin atmosphere.
- **Real sun position:** I calculated where the sun is overhead right now, so day and night on the globe match the real time.
- **Country hover:** I converted Natural Earth border data to lat/long and used ray casting and a point-in-polygon test to find the country under the mouse. The outline is drawn into a texture that the shader reads.
- **On-device speech to text:** I ran the Whisper model in the browser with Transformers.js (WebAssembly), so voice input works even in browsers without a speech service.
- **Neural text to speech:** I generated the reply lines with Microsoft's en-GB-RyanNeural voice.
- **Sound synthesis:** I built rain, brown noise and binaural beats from filters and oscillators, with no audio files.
- **REST APIs:** I used Open-Meteo and BigDataCloud, handled time zones, and cancelled old searches with AbortController.
- **Accessibility and performance:** I made an ARIA combobox with keyboard support, loaded heavy parts only when they come into view (IntersectionObserver), and added fallbacks when a permission is blocked.
- **Testing:** I tested every feature in a headless browser with Puppeteer, including a fake microphone and fake location.

8. Reflection

Using the APIs for a real purpose was the most useful part: Geolocation and Fetch give each user their own sunrise; Speech and Local Storage turn speaking into a saved list; Web Audio makes sound with no downloads. I also learned to plan for failure. A blocked permission, a missing speech service or a slow network all show a clear message or fall back to another method, so the page always works.